

PROJECT AND TEAM INFORMATION

Project Title

Cross-Broker Real-Time Portfolio Aggregation System

Student / Team Information

Team Name: Team #	Avengers
Team member 1 (Team Lead)	Singh, Uday Raj – 2510010552 udofficial789@gmail.com 
Team member 2	Jalal, Soumy – 2510370148 soumyj12@gmail.com 
Team member 3	Jain, Saksham – 2510370127 saksham82730@gmail.com 

PROPOSAL DESCRIPTION

Motivation

Investments are often distributed across multiple broker platforms. A user may hold different stocks through Groww and Zerodha, making it difficult to view the complete investment position from one place. This project proposes a Cross-Broker Real-Time Portfolio Aggregation System that collects permitted information from multiple brokers and presents it through one unified dashboard. The dashboard will display stock names, quantities, invested amount, current value, profit/loss and chronological trade history. The system will also collect current market information and relevant stock news. A simple one-year projection based on recent performance will provide an estimated future value for comparison. The project demonstrates practical applications of C++, Python, Object-Oriented Programming, pointers, dynamic memory, Inter-Process Communication, Linked Lists and Binary Search Trees.

State of the Art / Current solution

Broker applications such as Groww and Zerodha provide account-specific information such as holdings, transactions, market prices and profit/loss. However, users with investments across multiple brokers generally need to switch between applications to obtain a complete portfolio view. Some portfolio tracking solutions provide aggregation, but this project focuses on implementing the underlying data flow using two different programming languages. Python will handle network communication and data collection, while C++ will perform structured processing and portfolio management. A local TCP socket will connect the Python and C++ processes and provide the Inter-Process Communication (IPC) layer. The C++ program will receive the data stream, parse it and organize the information using OOP and custom data structures.

Project Goals and Milestones

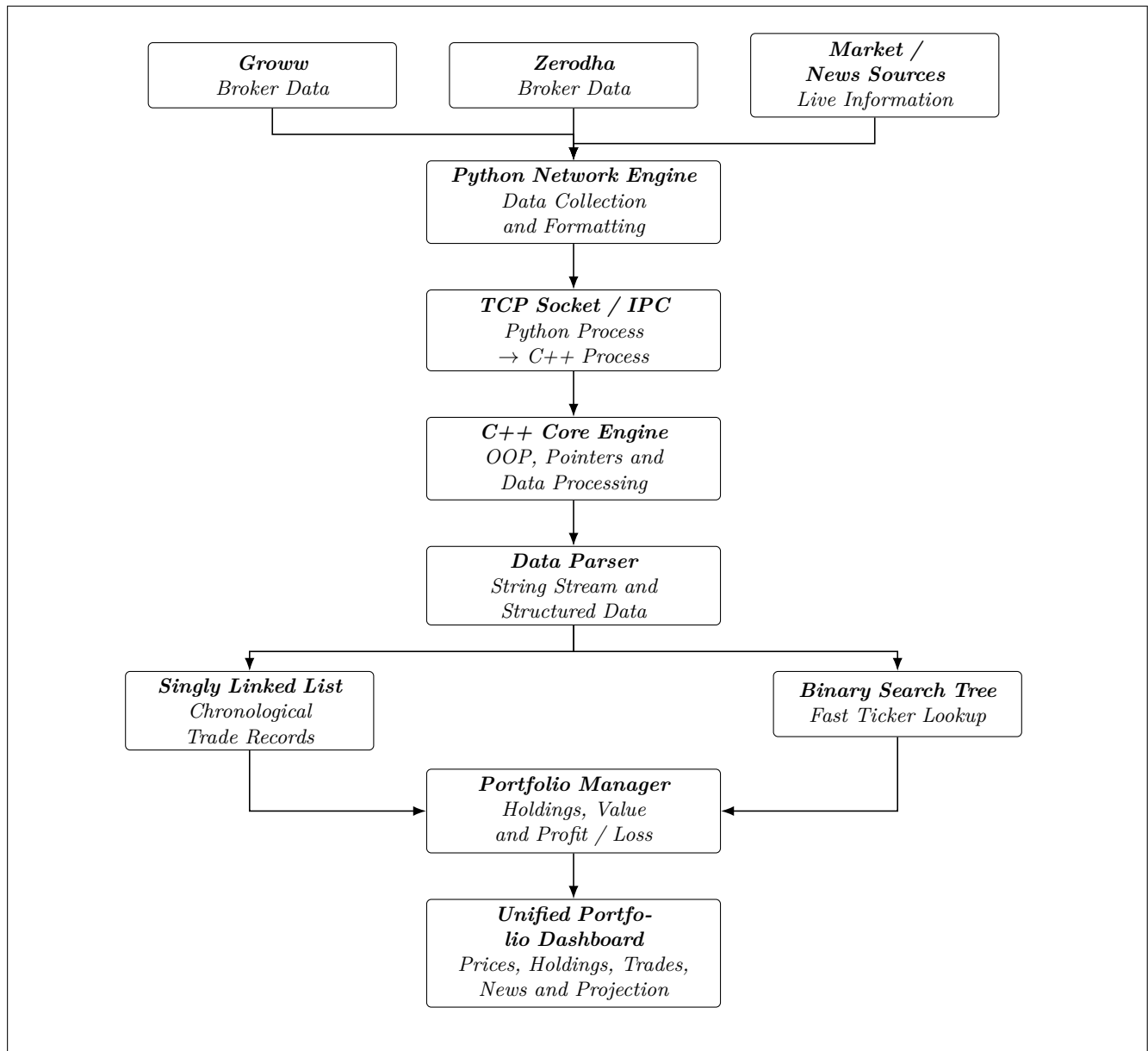
The main goal is to develop a unified system that collects, processes and displays investment information from multiple sources.

1. Identify the required broker, market and news information.
2. Develop the Python network layer for data collection.
3. Establish TCP socket communication between Python and C++.
4. Design the C++ OOP core for data processing.
5. Implement a parser for the received data stream.
6. Use a Singly Linked List for chronological trade records.
7. Use a Binary Search Tree for efficient ticker-based searching.
8. Calculate holdings, invested amount, current value and profit/loss.
9. Integrate relevant stock news and a basic one-year projection.
10. Provide broker-wise portfolio comparison.
11. Test the complete system using live or suitable sample data.

Project Approach

The proposed system will use a hybrid Python-C++ architecture, where each language performs a specific role. Python will act as the network and data collection layer. It will collect permitted broker information, market prices and relevant stock news and format the information for transmission. Python and C++ will operate as separate processes. A local TCP socket will provide the Inter-Process Communication (IPC) channel between them. This demonstrates how independent programs can exchange data in real time. The C++ program will receive the incoming byte stream and parse it using string-stream functionality. Classes will represent portfolio, trade and ticker information, providing encapsulation and organized data management. A Singly Linked List will store trade records in chronological order. A Binary Search Tree will organize ticker information and support efficient stock searches. Dynamic memory will be demonstrated using **new** and **delete**. Destructors will release dynamically allocated memory, while pointers will be used to manage dynamically created objects and linked structures. The processed information will be used to calculate holdings, invested amount, current value and profit/loss. The dashboard will additionally display market prices, news and a simple one-year estimated value based on recent performance. The main technologies are Python, C++, TCP sockets, OOP, pointers, dynamic memory, string streams, Linked Lists and Binary Search Trees.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The completed system will provide a unified dashboard for monitoring investments across multiple broker accounts. It will display broker-wise holdings, stock names, quantities, invested amount, current value, profit/loss and trade history. The dashboard will also show current market prices and relevant news for held stocks. A simple one-year estimated value based on recent performance will help users compare possible future values.

The main technical deliverables will include a Python network engine, TCP socket based IPC connection, C++ OOP core, data parser, Singly Linked List, Binary Search Tree, pointer-based operations and dynamic memory management.

The project will demonstrate how networking, Inter-Process Communication, Object-Oriented Programming and Data Structures can work together in a practical real-time application.

Assumptions

The system assumes that required broker, market and news information can be obtained through permitted APIs or suitable test data. Internet access will be required for external information. Authentication credentials will not be hard-coded into the program.

If live broker access is unavailable, representative sample data will be used to demonstrate the complete data flow. The one-year projection will be treated only as an approximate estimate based on recent performance and will not guarantee future returns.

The system is intended for portfolio monitoring and academic demonstration. It will not automatically execute buy or sell orders.

References

1. Python Documentation – Socket Programming.
2. C++ Reference – Classes, Destructors and Object-Oriented Programming.
3. C++ Reference – String Streams and Standard Library.
4. C++ Reference – Dynamic Memory Management.
5. Documentation for selected broker, market-data and news interfaces.